In all three cases, you set the CACHES value in your settings.py file to turn on caching, indicate your style of caching backend, and provide any additional parameters your backend requires. The following example is for a Memcached server running on port 11211 on the same machine as the Django instance:

```
# RiffMates/RiffMates/settings.py
...
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.memcached.PyMemcacheCache",
        "LOCATION": "127.0.0.1:11211",
    }
}
```

For more details on caching in Django, including the specific settings you need for different backends, see https://docs.djangoproject.com/en/dev/topics/cache/.

B.8 Putting it all together

If you think you're ready to go, look at the Django documentation's "Deployment Checklist": https://docs.djangoproject.com/en/dev/howto/deployment/checklist/.

If you are set up, and your infrastructure is all in place, there are a series of actions necessary for each deployment:

- Update your project code on the server.
- Install any third-party packages that were newly added to your project.
- Optionally, upgrade third-party packages.
- Copy static files to the production location.
- Sync the project's ORM with the production database.

B.8.1 Builds and releases

In the case of a PaaS, much of this is taken care of for you. For a VPS, you may want to write a script or two, ensuring a consistent process with each release. Some PaaS providers split the deployment process into two steps: *build* and *release*. When using containers, the build process is what builds the container object, while the release process pushes it into the infrastructure. Even if you're not using containers, thinking of deployment as a two-phase process can be beneficial.

If updating your code or any associated third-party libraries fails, you don't want to run the database migration command. Grouping the code updates, package updates, and static file management into a build script, while putting the migration call in a release script, means you won't be triggering changes in the database unless everything else went smoothly.

How you update the server's copy of your project code depends on your infrastructure. For simple systems, having a clone of your Git repo on the server means you simply do `git pull` on the branch used for production (typically `main` or `master`). For a PaaS like Heroku, this is done for you automatically. For more complex systems, with multiple servers being load balanced, you need to look into configuration management